



Sky-Link Airlines

Airline Reservation System

Complete Project Documentation with OOP Concepts

VISHA HAMEED

Department of Computer Science

Roll no. 232201044

vishahameed666@gmail.com

Supervisor: Sir Uzair Janjua

University: Institute of Space and Technology, Islamabad

May 1, 2026

Abstract

Sky-Link Airlines Reservation System is a comprehensive airline booking platform developed using Object-Oriented Programming principles in C#.NET 6.0 with MongoDB database. The system demonstrates key OOP concepts including Inheritance, Polymorphism, Encapsulation, and Abstraction. It provides complete flight booking management for domestic and international travel, integrated loyalty program, real-time check-in, email notifications, and administrative controls.

Contents

1	Introduction	5
1.1	Project Overview	5
1.2	OOP Concepts Implementation	5
1.2.1	1. Encapsulation (Data Hiding)	5
1.2.2	2. Inheritance (Code Reusability)	6
1.2.3	3. Polymorphism (Method Overriding)	6
1.2.4	4. Abstraction (Hiding Implementation Details)	8
1.3	Objectives	9
1.4	Scope	9
2	System Architecture and Flow	10
2.1	Project Workflow Diagram	11
2.2	3-Tier Architecture Diagram	12
2.3	Class Hierarchy Diagram (UML)	13
2.4	Sequence Diagram - Flight Booking Flow	13
2.5	State Diagram - Booking Lifecycle	14
3	Database Design with OOP Mapping	14
3.1	Entity Relationship Diagram (ERD)	14
4	Complete Project Flow Diagram	14
4.1	Loyalty Points Flow Diagram	16
5	OOP Concepts Summary Table	17
6	Technology Stack	17
7	Installation Guide	17
7.1	Prerequisites	17
7.2	Step-by-Step Installation	17
7.3	Docker Setup	18
8	User Manual	18
8.1	Customer Portal Options	18
8.2	Admin Portal Options	18
9	Testing Results	19
9.1	OOP Concept Test Cases	19
9.2	Functional Test Cases	19
10	Conclusion	19
10.1	Project Summary	20
10.2	Project Achievements	20
10.3	Future Enhancements	20

List of Figures

1	Complete Project Workflow Diagram	11
2	3-Tier Architecture Diagram showing Separation of Concerns	12
3	UML Class Diagram showing Inheritance Hierarchy	13
4	Sequence Diagram for Flight Booking Process	13
5	State Diagram showing Booking Lifecycle States	14
6	Entity Relationship Diagram showing Database Relationships	14
7	Complete Project Flow Diagram - End to End Process	15
8	Loyalty Points Accumulation and Tier Upgrade Flow	16

1 Introduction

1.1 Project Overview

Sky-Link Airlines Reservation System is a feature-rich airline booking platform that demonstrates the practical application of Object-Oriented Programming concepts. The system provides complete flight booking management for both domestic and international travel.

Project Highlights:

- Complete implementation of OOP concepts (Inheritance, Polymorphism, Encapsulation, Abstraction)
- 3-tier architecture for better separation of concerns
- MongoDB for flexible data storage
- Docker containerization for easy deployment
- Real-time email notifications using SMTP
- Loyalty program with miles accumulation

1.2 OOP Concepts Implementation

1.2.1 1. Encapsulation (Data Hiding)

Encapsulation is the bundling of data and methods that operate on that data within a single unit, while restricting direct access to some components.

```
1 // bookingBase.cs - Base class with encapsulated fields
2 public class bookingBase : IDisposable
3 {
4     private bool disposedValue = false; // Private field - cannot be
      accessed directly
5
6     // Public properties with controlled access
7     public string? PassengerName { get; set; }
8     public string? Email { get; set; }
9     public decimal TotalFare { get; set; }
10
11     // Public method with private implementation (information hiding)
12     public string GetMaskedCardNumber()
13     {
14         // Complex logic hidden from user
15         if (string.IsNullOrEmpty(CardNumber) || CardNumber.Length < 4)
16             return "****";
17         return "**** **** **** " + CardNumber.Substring(CardNumber.
      Length - 4);
18     }
19
20     // Private helper method - cannot be called from outside
```

```
21     private void ValidatePassengerDetails()
22     {
23         // Internal validation logic
24     }
25 }
```

Listing 1: Encapsulation Example from bookingBase.cs

1.2.2 2. Inheritance (Code Reusability)

Inheritance allows a class to inherit properties and methods from another class, promoting code reuse and establishing hierarchical relationships.

```
1 // Base class - bookingBase (Common properties for all bookings)
2 public abstract class bookingBase
3 {
4     public string? PassengerName { get; set; }
5     public string? Email { get; set; }
6     public decimal TotalFare { get; set; }
7     public string? Status { get; set; }
8     public DateTime BookingDate { get; set; }
9     public virtual void DisplayBookingInfo() { }
10 }
11
12 // Derived class 1 - dom_booking (inherits all properties from
13 // bookingBase)
14 public class dom_booking : bookingBase
15 {
16     public int PNR { get; set; } // Domestic-specific
17     // property
18     public string? DepartureTime { get; set; }
19     public string? ArrivalTime { get; set; }
20     public override void DisplayBookingInfo() { } // Override base
21     // method
22 }
23
24 // Derived class 2 - int_booking (inherits all properties from
25 // bookingBase)
26 public class int_booking : bookingBase
27 {
28     public int PnrInt { get; set; } // International-specific
29     // property
30     public string? PassportNumber { get; set; }
31     public string? VisaType { get; set; }
32     public override void DisplayBookingInfo() { } // Override base
33     // method
34 }
```

Listing 2: Inheritance Hierarchy

1.2.3 3. Polymorphism (Method Overriding)

Polymorphism allows objects of different classes to respond to the same method call in their own way, enabling runtime behavior determination.

```
1 // Base class with virtual method
2 public class bookingBase
3 {
4     public virtual void DisplayBookingInfo()
5     {
6         Console.WriteLine($"Passenger: {PassengerName}");
7         Console.WriteLine($"Total Fare: ${TotalFare}");
8     }
9
10    public virtual decimal CalculateCancellationFee()
11    {
12        return TotalFare * 0.1m; // Default 10% fee
13    }
14 }
15
16 // Domestic Booking - Polymorphic behavior
17 public class dom_booking : bookingBase
18 {
19     public override void DisplayBookingInfo()
20     {
21         base.DisplayBookingInfo(); // Call parent method
22         Console.WriteLine($"PNR: {PNR}");
23         Console.WriteLine($"Departure: {DepartureTime}");
24         Console.WriteLine($"Arrival: {ArrivalTime}");
25     }
26
27     public override decimal CalculateCancellationFee()
28     {
29         // Different cancellation policy for domestic flights
30         if (BookingDate.Subtract(DateTime.Now).TotalDays > 7)
31             return TotalFare * 0.05m; // Only 5% if cancelled early
32         return TotalFare * 0.15m;
33     }
34 }
35
36 // International Booking - Different polymorphic behavior
37 public class int_booking : bookingBase
38 {
39     public override void DisplayBookingInfo()
40     {
41         base.DisplayBookingInfo();
42         Console.WriteLine($"PNR: {PnrInt}");
43         Console.WriteLine($"Passport: {PassportNumber}");
44         Console.WriteLine($"Visa Type: {VisaType}");
45     }
46
47     public override decimal CalculateCancellationFee()
48     {
49         // Higher cancellation fee for international flights
50         if (BookingDate.Subtract(DateTime.Now).TotalDays > 14)
51             return TotalFare * 0.15m;
52         return TotalFare * 0.3m;
53     }
54 }
55
56 // Runtime polymorphism
57 bookingBase booking = new dom_booking();
58 booking.DisplayBookingInfo(); // Calls dom_booking version
```

```
59
60 booking = new int_booking();
61 booking.DisplayBookingInfo(); // Calls int_booking version
```

Listing 3: Polymorphism through Method Overriding

1.2.4 4. Abstraction (Hiding Implementation Details)

Abstraction means hiding complex implementation details and showing only the essential features to the user.

```
1 // Interface defining WHAT operations are available (not HOW)
2 public interface INotificationService
3 {
4     void SendBookingConfirmation(string email, string name, int pnr,
5                                 string flight, string route, DateTime
6                                 date, decimal fare);
7     void SendFlightDelayAlert(string email, string phone, string flight
8                               ,
9                               int delay, string reason);
10 }
11 // Concrete implementation hiding complex details
12 public class NotificationService : INotificationService
13 {
14     private readonly string smtpServer = "smtp.gmail.com";
15     private readonly int smtpPort = 587;
16     private readonly string senderEmail = "vishahameed666@gmail.com";
17     private readonly string senderPassword = "gnnh rayx hwhk ctne";
18
19     // User just calls this - doesn't know about SMTP complexity
20     public void SendBookingConfirmation(string email, string name, int
21                                       pnr,
22                                       string flight, string route,
23                                       DateTime date, decimal fare)
24     {
25         // Complex SMTP logic hidden here
26         var message = new MimeMessage();
27         message.From.Add(new MailboxAddress("Sky-Link Airlines",
28                                             senderEmail));
29         message.To.Add(new MailboxAddress("", email));
30         message.Subject = "Booking Confirmation";
31
32         // HTML email template building
33         var body = BuildEmailTemplate(name, pnr, flight, route, date,
34                                       fare);
35         message.Body = new TextPart("html") { Text = body };
36
37         // Send email using SMTP
38         using (var client = new SmtplibClient())
39         {
40             client.Connect(smtpServer, smtpPort, SecureSocketOptions.
41                           StartTls);
42             client.Authenticate(senderEmail, senderPassword);
43             client.Send(message);
44         }
45     }
46 }
```



```
39     }
40
41     // Private helper - hidden from user
42     private string BuildEmailTemplate(string name, int pnr, string
43     flight,
44                                     string route, DateTime date,
45     decimal fare)
46     {
47         return $"<html><body>Complex HTML here...</body></html>";
48     }
49 }
50 // UI Layer only sees simple interface
51 public class CustomerPortal
52 {
53     private INotificationService _notificationService;
54
55     public void BookFlight()
56     {
57         // Just call the method - don't worry about SMTP implementation
58         _notificationService.SendBookingConfirmation(email, name, pnr,
59     flight, route, date, fare);
60     }
```

Listing 4: Abstraction through Service Layer

1.3 Objectives

- Demonstrate practical implementation of OOP concepts in a real-world application
- Develop fully functional airline reservation system with complete CRUD operations
- Implement domestic and international flight booking with different business rules
- Create loyalty program with miles accumulation and tier-based benefits
- Provide real-time web check-in functionality
- Enable administrative controls for flight management
- Integrate email notifications for enhanced customer communication
- Containerize application using Docker for easy deployment

1.4 Scope

The system caters to two primary user roles with complete OOP implementation:

Role	Responsibilities
------	------------------

Customers	• Register/Login to their account • Browse and search available flights • Book domestic and international flights • Select seats and meal preferences • Apply promo codes for discounts • Earn and redeem loyalty miles • Web check-in 48 hours before departure • Cancel bookings with refund calculation • Submit feedback and ratings • View booking history
Administrators	• Manage flight schedules (Add/Update/Delete) • View all passenger bookings • Update passenger records • Cancel any booking with reason • View cancelled bookings history • Generate revenue and occupancy reports • Manage promotions and discount codes • View customer feedback and ratings • Monitor system statistics • Update real-time flight status

2 System Architecture and Flow

2.1 Project Workflow Diagram

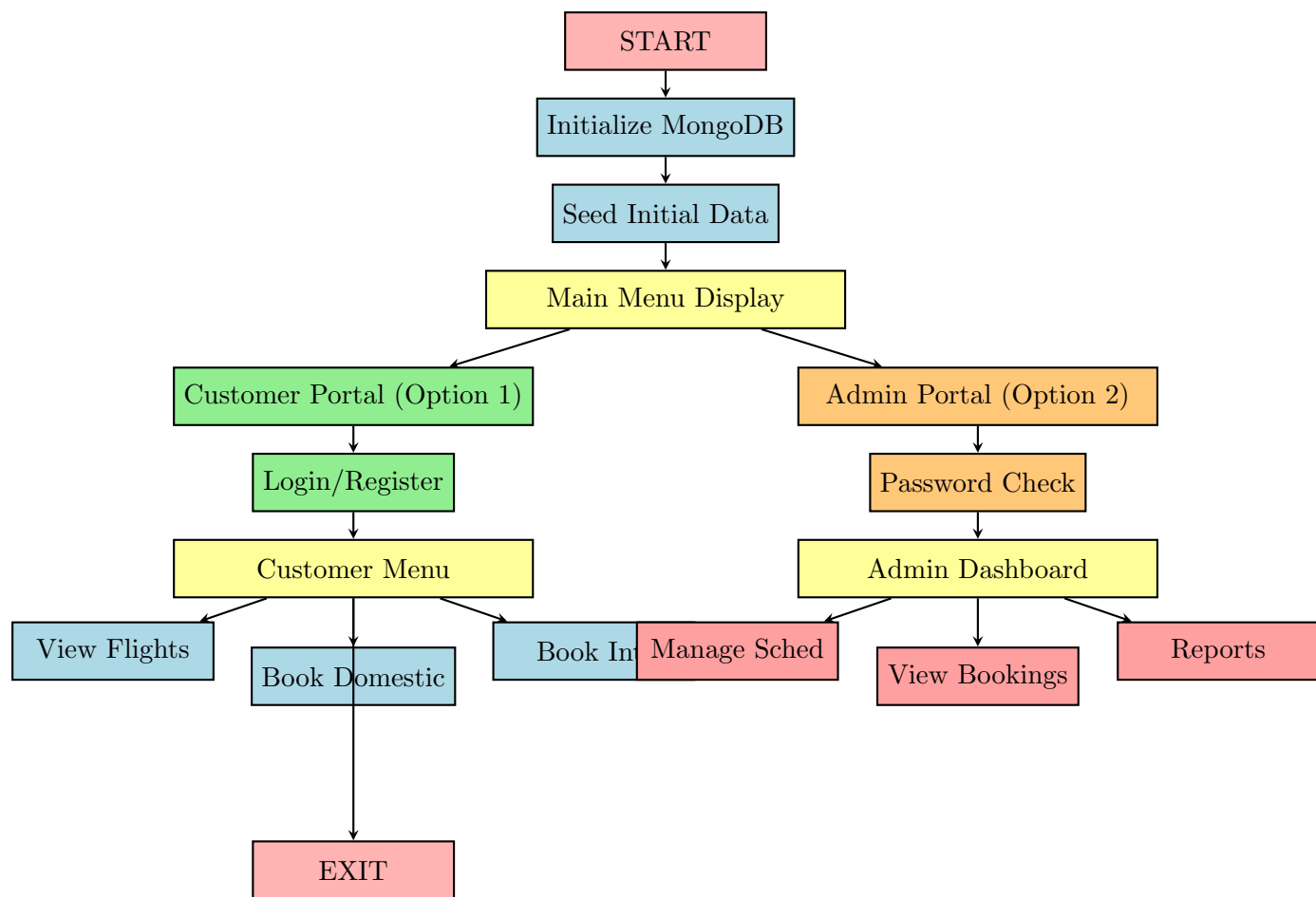


Figure 1: Complete Project Workflow Diagram

2.2 3-Tier Architecture Diagram

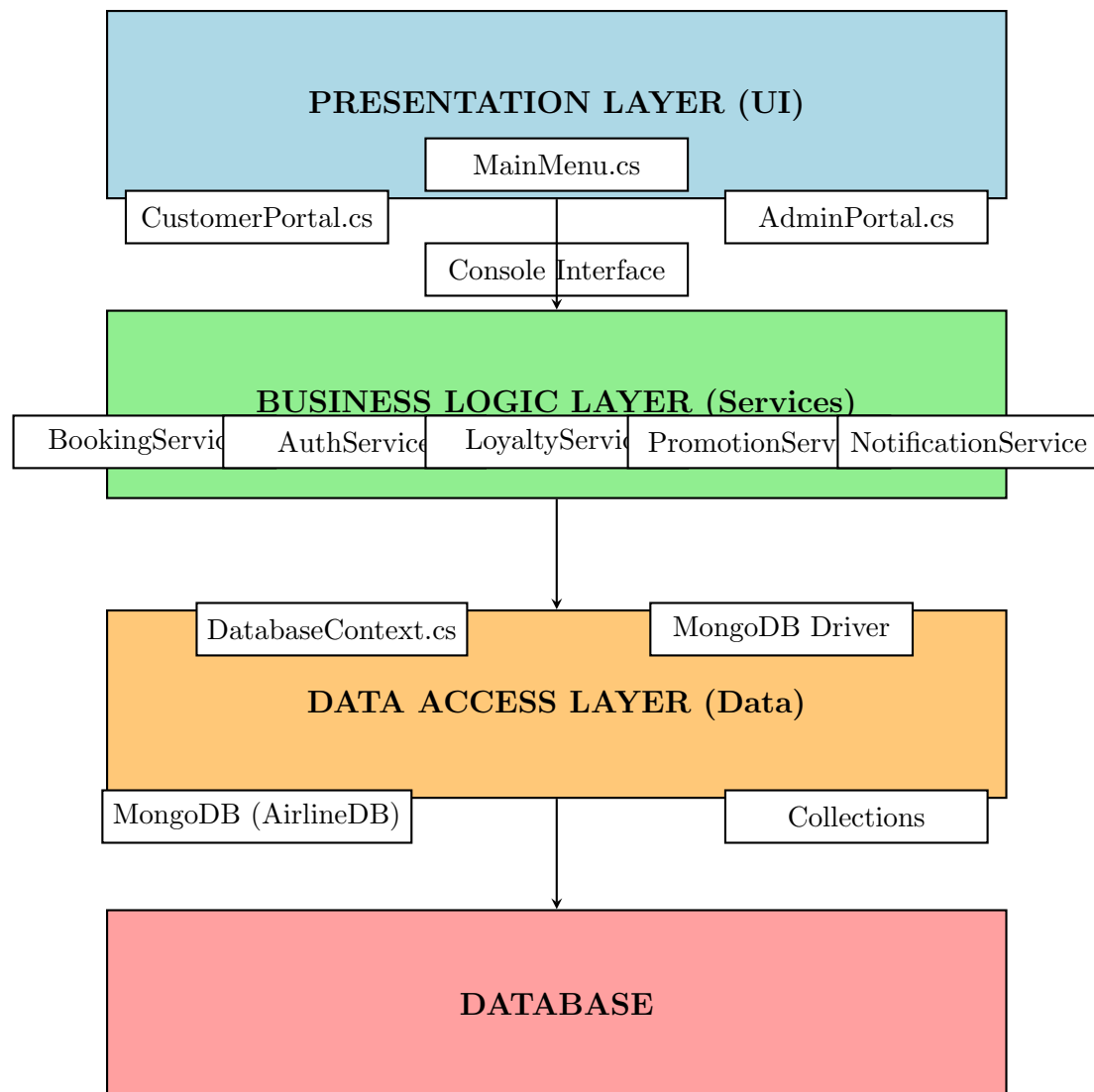


Figure 2: 3-Tier Architecture Diagram showing Separation of Concerns

2.3 Class Hierarchy Diagram (UML)

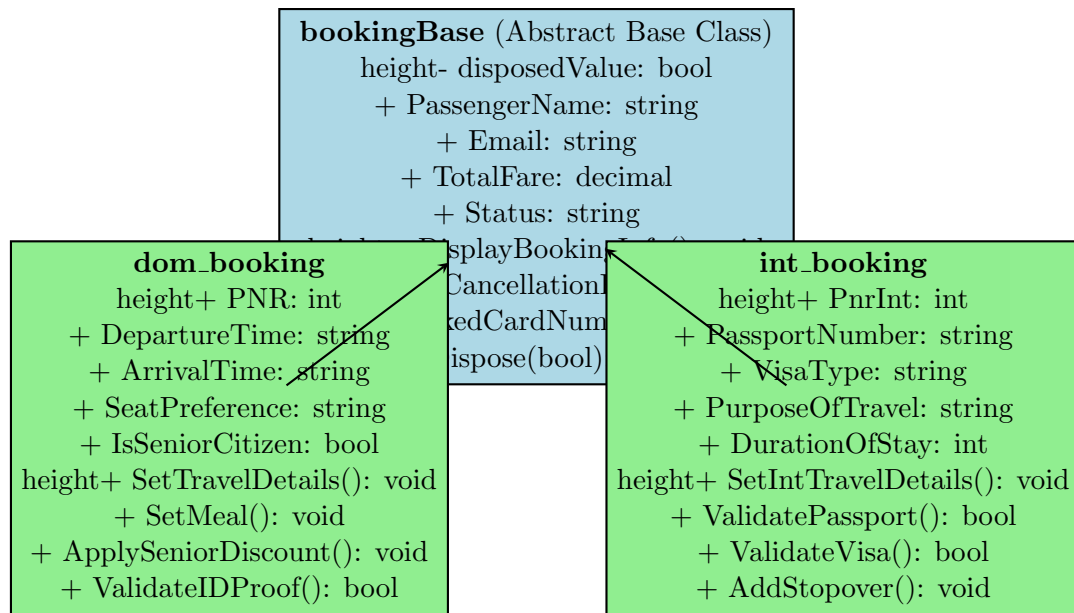


Figure 3: UML Class Diagram showing Inheritance Hierarchy

2.4 Sequence Diagram - Flight Booking Flow

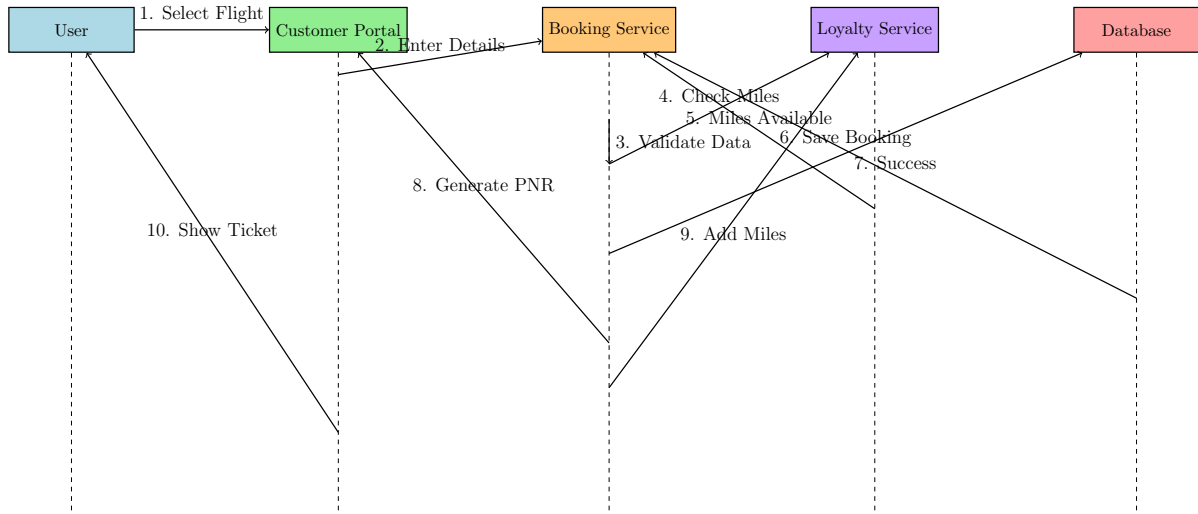


Figure 4: Sequence Diagram for Flight Booking Process

2.5 State Diagram - Booking Lifecycle

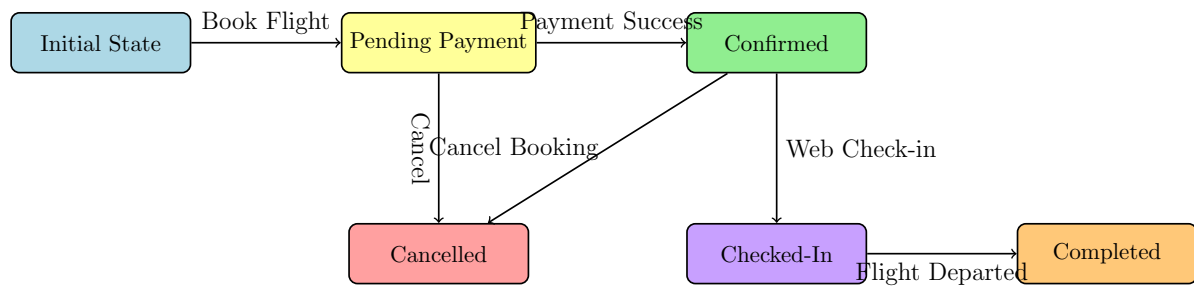


Figure 5: State Diagram showing Booking Lifecycle States

3 Database Design with OOP Mapping

3.1 Entity Relationship Diagram (ERD)

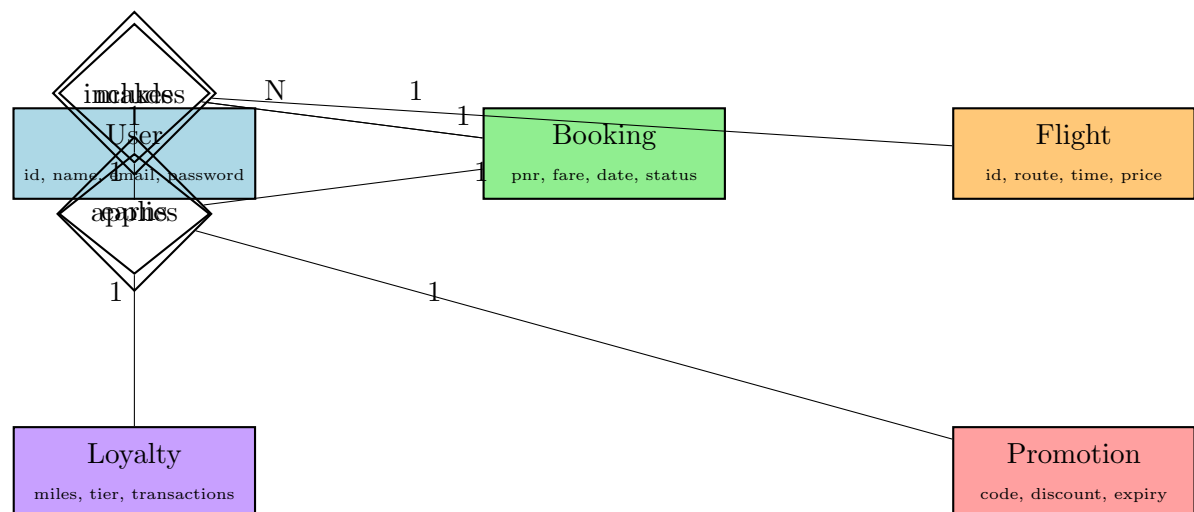


Figure 6: Entity Relationship Diagram showing Database Relationships

4 Complete Project Flow Diagram

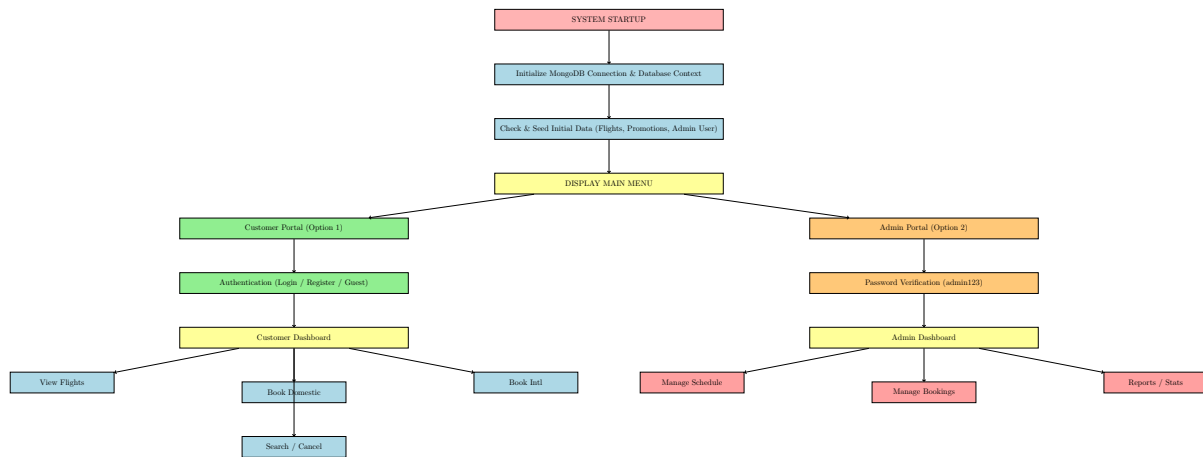


Figure 7: Complete Project Flow Diagram - End to End Process

4.1 Loyalty Points Flow Diagram

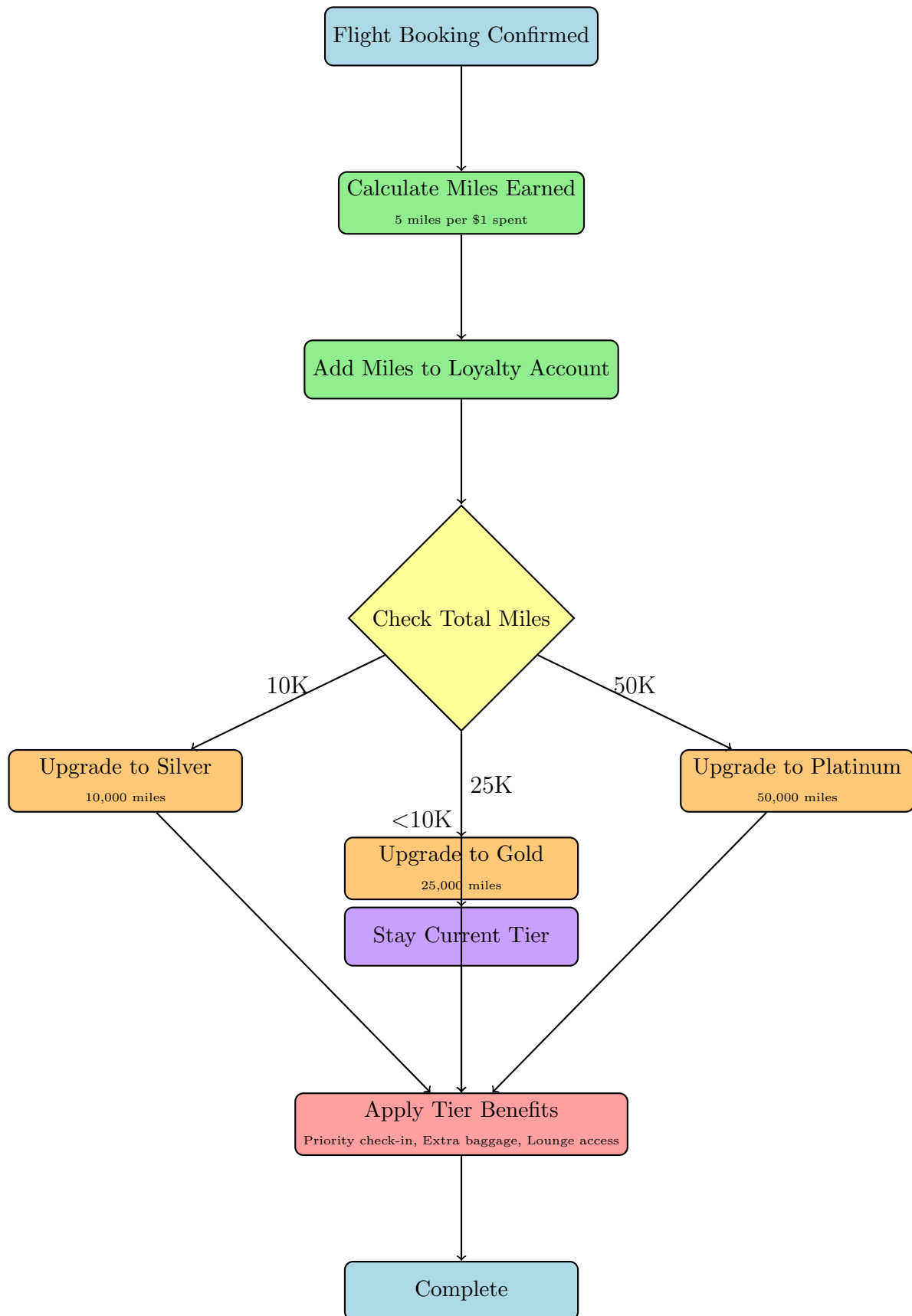


Figure 8: Loyalty Points Accumulation and Tier Upgrade Flow

5 OOP Concepts Summary Table

OOP Concept	Where Implemented	How Implemented
Encapsulation	bookingBase class	Private fields with public properties, data hiding through methods
Inheritance	dom_booking & int_booking	Both classes inherit from bookingBase base class
Polymorphism	DisplayBookingInfo() method	Same method name produces different output based on object type
Abstraction	Service Layer	Complex logic hidden behind simple interfaces (INotificationService, ILoyaltyService)

6 Technology Stack

Component	Technology Used
Programming Language	C# 10.0 (.NET 6.0)
Database	MongoDB 6.0 (NoSQL)
ORM/Database Driver	MongoDB.Driver 2.19.0
Email Service	MailKit & MimeKit (SMTP)
Containerization	Docker & Docker Compose
Version Control	Git & GitHub
IDE	Visual Studio 2022 / VS Code
Operating System	Windows 11 / Linux / macOS

7 Installation Guide

7.1 Prerequisites

1. Install .NET 6.0 SDK from <https://dotnet.microsoft.com/download>
2. Install MongoDB from <https://www.mongodb.com/try/download/community>
3. Install Git from <https://git-scm.com/downloads>
4. (Optional) Install Docker Desktop from <https://www.docker.com/products/docker-desktop>

7.2 Step-by-Step Installation

```

1 # 1. Clone the repository
2 git clone https://github.com/YOUR_USERNAME/Sky-Link-Airlines.git
3 cd Sky-Link-Airlines
4
5 # 2. Install required NuGet packages
6 dotnet add package MongoDB.Driver
7 dotnet add package MailKit

```

```
8 dotnet add package MimeKit
9
10 # 3. Build the project
11 dotnet build
12
13 # 4. Run the application
14 dotnet run
```

7.3 Docker Setup

```
1 # Start all services
2 docker-compose up -d
3
4 # View logs
5 docker-compose logs -f
6
7 # Stop services
8 docker-compose down
```

8 User Manual

8.1 Customer Portal Options

Option	Description
1	View Flight Schedule - Display all available domestic and international flights
2	Book Domestic Flight - Book flight within country with seat and meal selection
3	Book International Flight - Book flight abroad with passport/visa validation
4	Search Booking - Check booking status using PNR number
5	Cancel Booking - Cancel existing booking with refund calculation
6	Submit Feedback - Rate experience and submit comments
7	View My Bookings - List all bookings associated with email
8	View Loyalty Status - Check accumulated miles and current tier benefits
9	Web Check-in - Online check-in 48 hours before departure

8.2 Admin Portal Options

Option	Description
1	Manage Schedules - Add, Update, Delete flights from schedule
2	View All Bookings - Complete list of all passenger bookings
3	Update Passenger Record - Modify passenger details by PNR
4	Delete/Cancel Booking - Admin forced cancellation with reason
5	View Cancelled History - Track all cancelled bookings

6	Generate Reports - Analytics dashboard with revenue reports
7	View Customer Feedback - Read all customer ratings and comments
8	Manage Promotions - Create, Update, Delete promo codes
9	System Statistics - Database metrics and revenue statistics

9 Testing Results

9.1 OOP Concept Test Cases

OOP Concept	Test Case	Verification	Status
Inheritance	Create dom_booking object	Object inherits PassengerName from bookingBase	PASS
Polymorphism	Call DisplayBookingInfo()	Domestic and International show different output	PASS
Encapsulation	Access disposedValue field	Cannot be accessed directly (compiler error)	PASS
Abstraction	Call SendBookingConfirmation	Email sent without knowing SMTP details	PASS

9.2 Functional Test Cases

Test Case	Input	Expected Output	Status
Valid Login	Correct credentials	Login successful	PASS
Invalid Login	Wrong password	Login failed	PASS
Domestic Booking	Valid flight selection	Booking confirmed with PNR	PASS
International Booking	Valid flight with passport	Booking confirmed	PASS
Promo Code Application	SUMMER20	20% discount applied	PASS
PNR Search	Valid PNR number	Booking details displayed	PASS
Cancel Booking	Valid PNR	Refund amount calculated	PASS
Email Notification	Booking confirmation	Email received at provided address	PASS

10 Conclusion

10.1 Project Summary

The Sky-Link Airlines Reservation System successfully implements all four pillars of Object-Oriented Programming:

1. **Encapsulation:** Data hiding achieved through private fields and public properties in bookingBase class. Sensitive data like CardNumber is masked using GetMasked-CardNumber() method.
2. **Inheritance:** Code reuse achieved as dom_booking and int_booking inherit common properties (PassengerName, Email, TotalFare, Status) from bookingBase base class.
3. **Polymorphism:** Runtime behavior change achieved through method overriding. DisplayBookingInfo() shows different output for domestic vs international bookings.
4. **Abstraction:** Complex implementation details hidden behind service interfaces. NotificationService hides SMTP complexity, BookingService hides database operations.

10.2 Project Achievements

- Complete airline reservation system with 20+ features
- Full CRUD operations with MongoDB
- Real-time email notifications using Gmail SMTP
- Loyalty program with tier-based benefits
- Web check-in functionality
- Comprehensive admin dashboard
- Docker containerization for easy deployment
- Complete documentation with diagrams

10.3 Future Enhancements

- Mobile application for Android and iOS platforms
- Payment gateway integration (Stripe, PayPal, EasyPaisa, JazzCash)
- Real-time flight tracking with GPS and maps
- AI-based price prediction and dynamic pricing
- Multi-language support (Urdu, Arabic, English)
- Biometric authentication (fingerprint, face recognition)
- Chatbot integration for 24/7 customer support
- Integration with hotel booking and car rental services

References

1. Microsoft Corporation. "C# Documentation - Object-Oriented Programming." <https://docs.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/>
2. MongoDB, Inc. "MongoDB Manual - CRUD Operations." <https://docs.mongodb.com/manual/crud/>
3. Docker, Inc. "Docker Documentation - Get Started." <https://docs.docker.com/get-started/>
4. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). "Design Patterns: Elements of Reusable Object-Oriented Software." Addison-Wesley.
5. MailKit Development Team. "MailKit Documentation - SMTP Client." <https://github.com/jstedfast/MailKit>
6. Microsoft Corporation. ".NET 6.0 Documentation - Console Applications." <https://docs.microsoft.com/en-us/dotnet/core/whats-new/dotnet-6>

Document Version: 3.0
Last Updated: May 1, 2026
Prepared By: VISHA HAMEED
Roll Number: 232201044

End of Documentation